

双 STM32F103C8T6 环境安全监测系统

Dual-Node Environmental Safety Monitor

项目讲解 / Project Presentation

STM32F103C8T6 USART3 OLED Local Alarm

2026 年 5 月 29 日

- ① 项目目标与系统价值
- ② 硬件资源与引脚规划
- ③ 双节点架构与通信协议
- ④ 采集节点设计
- ⑤ 显示报警节点设计
- ⑥ 关键函数协作链路
- ⑦ 测试验证与演示流程
- ⑧ 总结与扩展方向

讲解主线

从“传感器数据如何产生”讲到“数据如何跨板传输”，最后落到“显示、报警、交互与记录如何协同”。

要解决的问题

- › 监测温湿度、空气质量、烟雾和火焰风险。
- › 将采集端和报警端分离，体现多 MCU 协作。
- › 在没有网络模块时，仍能完成本地闭环监测。

系统亮点

- › 两块 STM32 分布式架构。
- › USART3 自定义数据帧。
- › OLED、RGB、蜂鸣器、按键交互。
- › 可选 W25Q64 历史记录。

设计目标 / Design Goals

目标	设计要求	对应实现
稳定采集	每秒发帧，MQ 每帧采样，DHT11 安全间隔刷新	Sensor_App_Run() + 指数滑动平均
可靠通信	串口帧能识别错位和噪声	AA 55 帧头 + 长度 + checksum
本地报警	无需云端即可及时提示风险	RGB LED + 蜂鸣器 + OLED 状态页
易演示	可观察、可断线恢复、可用按键交互	USART1 日志、NODE LOST、 K1/K2
可扩展	后续可加 ACK、心跳、Flash 历史导出	协议字段、状态变量、可选 SPI Flash

板 A: SENSOR

- › STM32F103C8T6 双 USB 核心板
- › DHT11: 温湿度
- › MQ135: 空气质量模拟量
- › MQ2: 烟雾模拟量
- › 火焰模块: 数字触发

板 B: MONITOR

- › OLED: 状态与数值显示
- › 板载 RGB: 状态灯
- › 蜂鸣器: 声音报警
- › K1/K2: 切页、静音、阈值档位
- › W25Q64: 可选历史记录

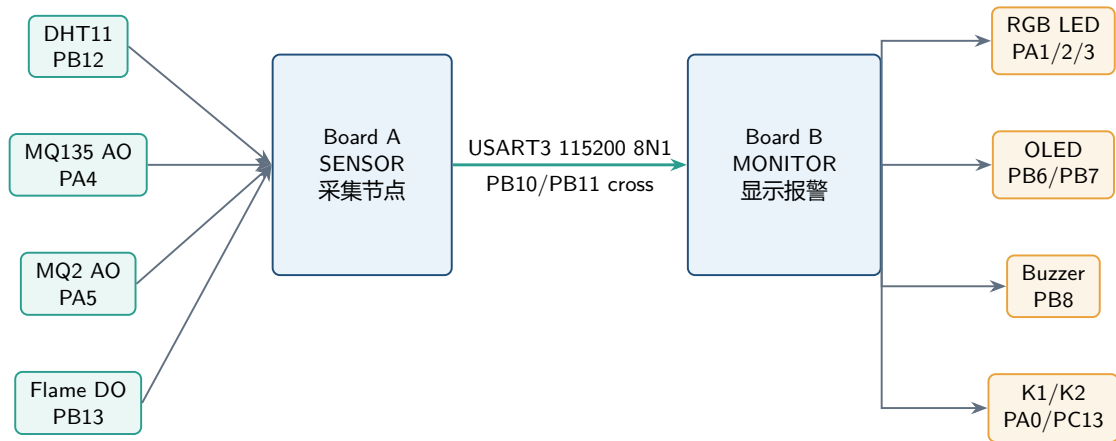
板载资源约束 / Board Resource Constraints

引脚	板载连接	本项目处理方式
PA9 / PA10	CH340C USB 转串口	保留 USART1 做 printf 调试
PA11 / PA12	USB Device	保留, 不分配外设
PA13 / PA14	SWD 下载调试	保留给调试器
PA1 / PA2 / PA3	板载 RGB LED	板 B 用于报警状态
PA0 / PC13	板载 K1 / K2	板 B 用于页面、静音、阈值
PB10 / PB11	普通可用 IO / USART3	用作双板通信链路

关键取舍

不使用 USART2 PA2/PA3, 是因为 PA2/PA3 已被板载 RGB 占用; 改用 USART3 PB10/PB11 更干净。

总体架构 / Overall Architecture



双固件构建方式 / One Source, Two Images

核心思想

同一份 Core/Src/main.c 通过 APP_NODE_ROLE 生成两个不同固件：

- › SensorDebug：板 A 采集节点
- › MonitorDebug：板 B 显示报警节点

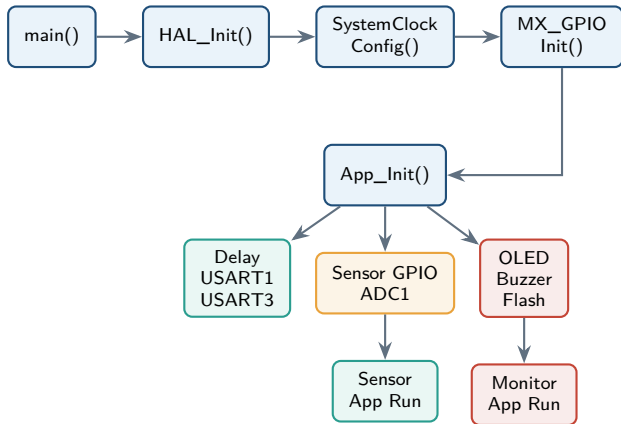
```
#define APP_ROLE_SENSOR 1
#define APP_ROLE_MONITOR 2

#if APP_NODE_ROLE == APP_ROLE_SENSOR
    Sensor_App_Run();
#else
    Monitor_App_Run();
#endif
```

收益

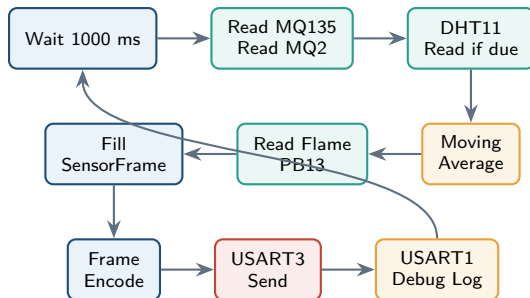
协议、工具函数、全局数据结构只维护一份，减少“发送端改了、接收端忘改”的风险。

启动链路 / Startup Path



公共外设先初始化，角色专属外设后初始化，逻辑
分层清晰，便于定位问题。

板 A 采集节点主循环 / Sensor Super-Loop



设计重点

每秒发帧，不阻塞显示端；MQ 每帧采样并做整数滑动平均，DHT11 按 $>2s$ 间隔刷新，错误通过状态位上报。

DHT11 单总线时序 / DHT11 Timing

- ❶ MCU 将 PB12 配成开漏输出。
- ❷ 拉低数据线约 20 ms 发起读取。
- ❸ 释放数据线，切换为输入。
- ❹ 等待 DHT11 响应低/高/低脉冲。
- ❺ 读取 40 bit 数据并校验。

函数拆分

- DHT11_SetOutput()
- DHT11_SetInput()
- DHT11_WaitLevel()
- DHT11_Read()

鲁棒性

DHT11 失败不会卡死系统，而是设置 STATUS_DHT_ERROR，让板 B 显示预警状态。

ADC 通道

- › MQ135 AO: PA4 / ADC1_CH4
- › MQ2 AO: PA5 / ADC1_CH5
- › ADC1 时钟: $72 \text{ MHz} / 6 = 12 \text{ MHz}$

指数滑动平均

$$avg_{new} = \frac{3 \times avg_{old} + raw}{4}$$

- › 不保存数组窗口。
- › 计算量小, 适合 STM32F103。
- › 平滑噪声但保留变化趋势。

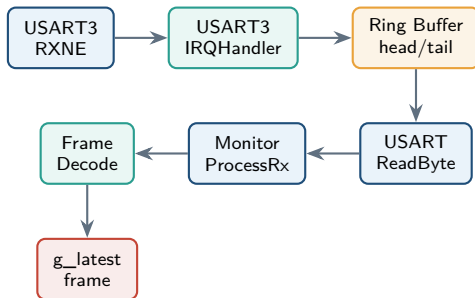
串口数据帧协议 / USART3 Frame Protocol

Index	Field	Meaning
0-1	AA 55	Frame header, used for resynchronization
2	LEN	Payload length, fixed to 9
3-4	TEMP/HUMI	DHT11 temperature and humidity
5-8	MQ135/MQ2	ADC readings, high byte first
9	FLAME	1 means flame detected
10	SEQ	Rolling frame number
11	STATUS	bit0 = DHT11 error
12	CHECKSUM	Low 8 bits of LEN + payload

核心函数

Frame_Encode() 负责结构体转字节帧, Frame_Decode() 负责帧头、长度、校验和验证。

板 B 接收设计 / Interrupt + Ring Buffer



为什么这样设计

中断只收字节，主循环解析完整帧。这样 ISR 短小稳定，OLED 刷新时也不容易漏掉串口数据。

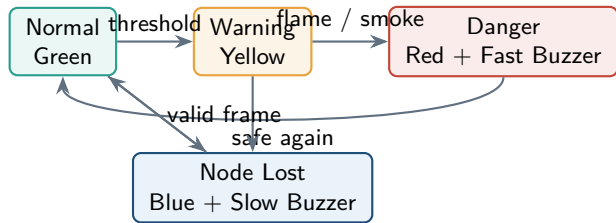
板 B 协作式调度 / Monitor Scheduler

任务	周期	函数与作用
串口解析	每轮循环	Monitor_ProcessRx(), 尽快消化接收缓冲
按键扫描	每轮循环	Monitor_UpdateButtons(), 保证 K1/K2 响应
报警输出	100 ms	Monitor_UpdateAlarm(), 更新 LED 和蜂鸣节奏
OLED 刷新	300 ms	Monitor_UpdateDisplay(), 控制刷新负担
Flash 记录	10 s	Flash_LogFrame(), 降低写入频率

无 RTOS 的处理方式

使用 HAL_GetTick() 做时间片调度, 避免长阻塞, 让串口、显示、按键、报警共存。

报警状态机 / Alarm State Machine



优先级: Danger > Node Lost > Warning >

Normal。静音只影响蜂鸣器, 不改变 LED 状态。

Page 0: 监测页

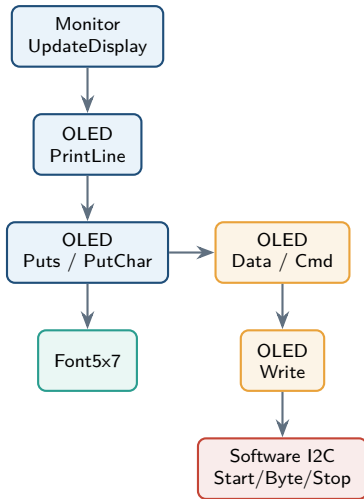
- › 系统状态: NORMAL / WARN / DANGER / NODE LOST
- › 温度、湿度
- › MQ135 空气质量 ADC
- › MQ2 烟雾 ADC 与火焰状态

Page 1: 参数页

- › 当前阈值档位
- › 空气质量预警阈值
- › 烟雾预警/危险阈值
- › 帧序号与 Flash 状态

K1 切换页面, K2 短按静音, K2 长按切换阈值档位。

OLED 驱动分层 / OLED Driver Stack



按键	操作	系统反馈
K1	短按	<code>g_page</code> 翻转, OLED 在监测页与参数页之间切换
K2	短按	<code>g_mute_until_ms = now + 60000</code> , 蜂鸣器静音 60 秒
K2	长按	<code>g_threshold_profile</code> 递增, 切换阈值档位

边沿检测

使用 `k1_last`、`k2_last` 保存上一次电平, 只在按下或松开边沿触发动作, 避免按住时重复执行。

初始化策略

- › SPI2 使用 PB13/PB14/PB15。
- › PB12 作为 CS。
- › 读取 JEDEC ID 判断芯片是否存在。
- › 未检测到芯片时主流程继续运行。

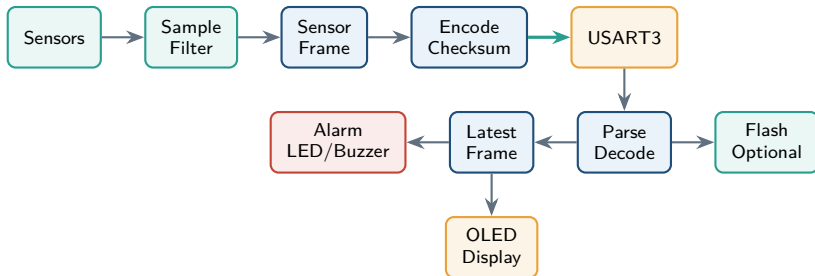
记录内容

- › 帧序号与状态位
- › 温湿度、MQ ADC、火焰状态
- › 当前报警状态
- › HAL_GetTick() 时间戳
- › 阈值档位与 checksum

关键全局状态 / Shared State Variables

变量	写入者	读取者与作用
g_latest_frame	Monitor_ProcessRx()	OLED、报警、Flash 使用的最新合法数据
g_last_rx_ms	Monitor_ProcessRx()	Monitor_NodeLost() 判断离线
g_page	Monitor_UpdateButtons()	OLED 页面选择
g_threshold_profile	Monitor_UpdateButtons()	影响报警阈值与显示参数
g_mute_until_ms	Monitor_UpdateButtons()	控制蜂鸣器静音截止时间
g_flash_present	Flash_Init_Custom()	控制是否启用日志记录
rx_buf/head/tail	ISR / parser	USART3 接收环形缓冲区, 负责中断与主循环之间的数据交接

端到端数据路径 / End-to-End Data Path



异常	检测位置	处理方式
DHT11 读取失败	DHT11_Read()	设置 STATUS_DHT_ERROR, 继续发帧
串口噪声或错位	Frame_Decode()	丢弃坏帧, 重新寻找 AA 55
接收缓冲满	USART3_IRQHandler()	丢弃最新字节, 避免 ISR 阻塞
板 A 离线	Monitor_NodeLost()	OLED 显示 NODE LOST, 蓝灯慢蜂鸣
Flash 未接入	Flash_Init_Custom()	g_flash_present=0, 主功能不受影响
时钟初始化失败	SystemClock_Config()	Error_Handler() 停机保护

单节点测试

- › 板 A: DHT11、MQ135、MQ2、火焰模块。
- › 板 B: OLED、RGB、蜂鸣器、K1/K2。
- › USART1 日志: 115200 8N1。

系统测试

- › USART3 双板通信。
- › 断线后 NODE LOST。
- › 烟雾/火焰触发报警。
- › 任意节点重启恢复。

演示流程 / Demonstration Flow

- ① 烧录 Fire_F103_sensor.hex 到板 A, 烧录 Fire_F103_monitor.hex 到板 B。
- ② 连接 USART3: A PB10 到 B PB11, B PB10 到 A PB11, 并共地。
- ③ 打开两个 CH340C 串口, 确认板 A [SENSOR] 日志和板 B [MONITOR] rx 日志。
- ④ 展示 OLED 监测页和参数页, 按 K1 切换页面。
- ⑤ 触发烟雾/火焰模块, 观察状态灯和蜂鸣器变化。
- ⑥ 短按 K2 展示静音, 长按 K2 展示阈值档位切换。
- ⑦ 断开 USART3, 展示 NODE LOST 自动检测。

```
cmake --preset SensorDebug  
cmake --build --preset SensorDebug  
  
cmake --preset MonitorDebug  
cmake --build --preset MonitorDebug
```

文档结构

- › README.md / README.zh-CN.md
- › WIRING.md
- › docs/FUNCTION_GUIDE.md
- › docs/FUNCTION_DESIGN_WALKTHROUGH.*.md
- › docs/PROJECT_STRUCTURE.md

限制	说明
MQ 数值未标定 ppm 通信仍以单向上报为主 OLED 文本较简单 Flash 记录为演示型	当前使用原始 ADC 值，需要实测标定曲线才能换算浓度 预留了 B 到 A 的 ACK/心跳方向，但基础版本先保证 A 到 B 稳定 为保证 0.96 寸屏幕可读性，使用 4 行英文/数字状态文本 使用第 0 扇区循环写，不是工业级 wear leveling

功能扩展

- › B 到 A ACK / 心跳。
- › 阈值写入 W25Q64, 重启保持。
- › 增加光敏、人体红外等传感器。
- › 历史记录导出到 PC。

工程改进

- › 拆分 main.c 为 protocol、sensor、monitor 模块。
- › 增加 CRC8/CRC16。
- › 更完整的按键消抖。
- › 增加单元测试或串口帧模拟器。

项目成果

本系统完成了双 STM32 节点分工、USART3 数据通信、本地 OLED 显示、RGB/蜂鸣器报警、按键交互和可选 Flash 记录。

工程价值

项目体现了嵌入式系统中的资源规划、通信协议设计、状态机、协作式调度和异常恢复思路。

谢谢观看 / Thank you